

PROYECTO PRX

Informe de defensa del Sprint 1

Notas, Bitácora y Autenticación

Documento técnico para presentar y defender el trabajo implementado, explicar dónde se encuentra cada pieza de código y relacionar las decisiones de arquitectura con los principios SOLID y los patrones empleados.

Dato	Detalle
Proyecto	PRX
Entrega	Presentación y defensa del Sprint 1
Fecha de preparación	22 de septiembre de 2026
Fecha límite informada	23 de septiembre de 2026 a las 23:59
Tecnologías principales	Angular 21, PrimeNG 21, NestJS 11, CQRS, Prisma 6 y MySQL

Conclusión ejecutiva

Se construyó una base funcional y modular para autenticación, Notas y Bitácora. Notas permite consultar, crear, visualizar y editar contenido, archivos, imágenes, enlaces y tareas; Bitácora permite registrar actividades personales con tareas y enlaces, separar la consulta de la edición y marcar tareas directamente desde la vista. La autenticación protege rutas y API mediante JWT, sesiones, renovación de tokens y roles. Frente al backlog original, el Sprint 1 aún tiene brechas concretas: falta un dashboard autenticado, falta búsqueda en el listado de notas y la creación inicial de una nota todavía no incorpora tareas.

Contenido

1. Alcance y resultado del Sprint 1
 2. Arquitectura y tecnologías utilizadas
 3. Módulo de Notas
 4. Módulo de Bitácora
 5. Autenticación y autorización
 6. Aplicación de SOLID
 7. Patrones de diseño
 8. Calidad, puntualidad y evidencias
 9. Guion para la presentación y defensa
 10. Preguntas probables y respuestas
 11. Conclusiones y trabajo pendiente
- Anexo. Inventario de archivos principales

Fuentes de evidencia

- Código fuente actual de prx-frontend y prx-backend.
- Product_Backlog_PRX_Mantenimiento_Evolutivo.xlsx, hojas PRODUCT BACKLOG PRX y RESUMEN DE SPRINTS.
- Compilación y comprobaciones locales realizadas el 22 de septiembre de 2026.

1 Alcance y resultado del Sprint 1

El objetivo registrado para el Sprint 1 es crear la nueva estructura visual del sistema, el Home y la gestión básica de notas. El backlog asigna tres historias de usuario y 18 puntos. El análisis siguiente compara esas historias con el comportamiento que existe actualmente en el repositorio.

Historia	Alcance esperado	Evidencia actual	Estado técnico
HU 01	Pantalla principal para acceder a los módulos	Existe navegación y una landing pública, pero no un dashboard principal específico para el usuario autenticado.	Pendiente
HU 02	Listado, búsqueda y vista previa de notas	Existen tarjetas, vista previa, paginación, autor, adjuntos y acceso al detalle. No existe búsqueda de notas.	Parcial
HU 03	Crear y modificar notas con tareas y contenido	Creación y edición de título, contenido, archivos, imágenes y enlaces. Las tareas se gestionan en edición, pero no durante la creación inicial.	Parcial

Resultado: el Sprint 1 tiene una base funcional importante, pero no debe declararse completamente terminado hasta cerrar las tres brechas indicadas.

Bitácora aparece en el backlog como trabajo de un sprint posterior. Incluirla en la demostración es válido como avance adicional, siempre que se aclare que no sustituye los criterios pendientes del Sprint 1.

Criterio para defender el avance

- Mostrar primero el flujo funcional que sí está terminado.
- Relacionar cada pantalla con su endpoint y persistencia.
- Explicar las decisiones de arquitectura y seguridad con ejemplos reales del código.
- Reconocer las brechas como trabajo pendiente priorizado, sin presentarlas como completadas.

2 Arquitectura y tecnologías utilizadas

La solución separa responsabilidades tanto en frontend como en backend. Esta separación permite cambiar la interfaz, la fuente de datos o las reglas de negocio sin concentrar todo en un único componente.

Capa	Frontend Angular	Backend NestJS
Presentación	Páginas, plantillas, estilos, rutas y componentes standalone.	Controladores HTTP, pipes, guards y decoradores.
Aplicación	Facades, formularios y coordinación del estado.	Commands, Queries y Handlers mediante CQRS.
Dominio	Modelos, requests y contratos de API.	Entidades y contratos abstractos de repositorio.
Infraestructura	Servicios HTTP, configuración de endpoints y stores.	Repositorios Prisma, mappers, JWT, bcrypt y almacenamiento.

Flujo general de una operación

1. La persona interactúa con una página Angular.
2. El componente delega la operación a una Facade.
3. La Facade utiliza un servicio API y actualiza el Store.
4. El controlador NestJS recibe y valida la solicitud.
5. CommandBus o QueryBus dirige la operación al Handler correspondiente.
6. El Handler aplica permisos y reglas de negocio mediante contratos de repositorio.
7. La implementación Prisma guarda o consulta los datos y un Mapper forma la respuesta.

Tecnologías

Tecnología	Uso en PRX
Angular 21	Interfaz web, rutas protegidas, formularios reactivos, signals y carga diferida.
PrimeNG 21	Botones, tarjetas, etiquetas, confirmaciones, skeletons y controles visuales.
RxJS	Flujos asíncronos, manejo de loading, tap, catchError y finalize.
NestJS 11	API modular, inyección de dependencias, validación y seguridad.
NestJS CQRS	Separación entre comandos que modifican datos y consultas que los leen.
Prisma 6	Mapeo y acceso tipado a MySQL.
JWT y bcrypt	Tokens firmados y verificación segura de contraseñas.
Swagger	Descripción de endpoints y cuerpos multipart/form-data.

3 Módulo de Notas

Notas permite trabajar dentro de un repositorio normal o del repositorio íntimo del usuario. La solución actual cubre listado, paginación, vista previa, detalle, edición, eliminación lógica, archivos, imágenes, enlaces y tareas.

3.1 Funciones implementadas

- Listado por repositorio con tarjetas y paginación.
- Vista previa limitada a 180 caracteres para facilitar el escaneo.
- Creación con título, contenido, archivos, imágenes y enlaces web.
- Detalle independiente de la edición.
- Edición de título, contenido, tareas, enlaces y archivos conservados.
- Marcado de tareas desde la vista con guardado inmediato.
- Descarga de archivos mediante URL autorizada.
- Eliminación lógica y control de permisos según propietario o miembro del repositorio.

3.2 Modelo compartido en el frontend

Archivo: prx-frontend/src/app/features/notes/domain/models/note.model.ts

```
export interface NoteTaskModel {
  id: string;
  title: string;
  completed: boolean;
}

export interface NoteLinkModel {
  id: string;
  url: string;
}
```

Estas interfaces tipan las tareas y enlaces que viajan entre la pantalla, la Facade y la API. El identificador permite actualizar o eliminar un elemento concreto; completed representa el estado de la tarea; url almacena el enlace validado.

3.3 Comunicación con la API

Archivo: prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts

```
formData.append('title', data.title);
formData.append('content', data.content);
formData.append('tasks', JSON.stringify(data.tasks));
formData.append('links', JSON.stringify(data.links));
formData.append('retainedFileIds', data.retainedFileIds.join(','));
```

La actualización usa FormData porque una misma solicitud puede transportar texto estructurado y archivos binarios. Las tareas y enlaces se serializan como JSON; retainedFileIds indica cuáles adjuntos existentes deben conservarse.

3.4 Marcado de tareas desde la vista

Archivo: prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.ts

```
protected toggleTask(task: NoteTaskModel): void {
  this.tasks.update((tasks) =>
    tasks.map((item) => item.id === task.id
      ? { ...item, completed: !item.completed }
      : item),
  );
  if (!this.editing()) this.persist(false);
}
```

El estado se actualiza de forma inmutable. Cuando la pantalla está en modo consulta, persist(false) envía el cambio sin abrir ni cerrar el editor. Esta decisión reproduce el comportamiento esperado de una lista de tareas: marcar debe ser una acción rápida.

3.5 Endpoints y backend

Método	Ruta	Responsabilidad
GET	/prx/notes/repository/:repositoryId	Listar notas paginadas del repositorio autorizado.
GET	/prx/notes/:id	Consultar el detalle de una nota.
POST	/prx/notes/repository/:repositoryId	Crear nota y hasta cinco archivos.

Método	Ruta	Responsabilidad
PUT	/prx/notes/:id	Actualizar contenido, tareas, enlaces y adjuntos.
DELETE	/prx/notes/:id	Realizar eliminación lógica.
GET	/prx/notes/files/:id/download	Obtener una URL de descarga autorizada.

Archivo: prx-backend/src/modules/notes/application/commands/update-note/update-note.handler.ts

```
const tasks = this.parseTasks(command.dto.tasks);
const links = this.parseLinks(command.dto.links);

if (retainedFiles.length + command.files.length > 5) {
  throw new BadRequestException('La nota permite un máximo de 5 archivos');
}
```

El Handler no confía en el JSON recibido. Convierte y valida tareas y enlaces, limita cantidades y comprueba permisos antes de persistir. También sincroniza los adjuntos: elimina los descartados del repositorio y del almacenamiento, y carga los nuevos.

3.6 Persistencia

Archivo: prx-backend/prisma/schema.prisma, modelo Note

```
model Note {
  id          Int          @id @default(autoincrement())
  repositoryId Int
  title       String
  content     String
  tasks       Json?
  links       Json?
  status      Int          @default(1)
}
```

tasks y links se guardan como JSON porque son listas pequeñas que pertenecen a una nota y se recuperan junto con ella. status permite eliminación lógica, conservando trazabilidad. Los archivos se normalizan en NoteFile porque requieren metadatos y ciclo de vida propio.

3.7 Brechas actuales de Notas

Pendiente 1: agregar búsqueda por título o contenido en la página de listado y soportarla en el DTO y repositorio del backend.

Pendiente 2: permitir agregar tareas durante la creación inicial; CreateNoteRequest actualmente transporta enlaces, pero no tareas.

Mejora recomendada: añadir pruebas unitarias de parseTasks, parseLinks, permisos y persistencia inmediata de checkboxes.

4 Módulo de Bitácora

Bitácora registra actividades personales del usuario estándar. La implementación mantiene los datos aislados por userId y expone únicamente las bitácoras propias. Aunque corresponde a un sprint posterior en el backlog, se documenta porque está implementada y forma parte de la demostración solicitada.

4.1 Funciones implementadas

- Crear una bitácora con título, descripción, tareas y enlaces.
- Listar únicamente las bitácoras del usuario autenticado.
- Abrir una vista de consulta mediante el botón Ver.
- Entrar a edición desde la vista mediante un botón independiente.
- Agregar o eliminar tareas y enlaces en edición.
- Marcar tareas desde la vista y guardar inmediatamente.
- Abrir enlaces externos de forma segura con noopener noreferrer.
- Eliminar mediante borrado lógico.

4.2 Separación entre ver y editar

Archivos: `binnacle-page.component.html` y `binnacle-detail-page.component.html`

```
<p-button
  label="Ver"
  icon="pi pi-eye"
  (onClick)="openBinnacle(binnacle.id)">
</p-button>

@if (!editing()) {
  <button (click)="startEditing()">Editar</button>
}
```

La lista ya no mezcla dos intenciones distintas. Ver navega a una lectura limpia; Editar aparece dentro del detalle. Esto reduce errores de uso y mantiene una jerarquía clara de acciones.

4.3 Guardado inmediato de tareas

Archivo: `prx-frontend/src/app/features/binnacles/presentation/pages/binnacle-detail-page.component.ts`

```
protected toggleTask(id: string): void {
  if (this.saving()) return;
  this.tasks.update((tasks) =>
    tasks.map((task) => task.id === id
      ? { ...task, completed: !task.completed }
      : task),
  );
  if (!this.editing()) this.persist(false);
}
```

saving evita solicitudes simultáneas. La actualización local ofrece respuesta inmediata y persist(false) envía título, contenido, tareas y enlaces al endpoint PUT sin sacar al usuario de la vista. Si la solicitud falla, el componente restaura el objeto recibido previamente.

4.4 Seguridad y propiedad

Archivo: `prx-backend/src/modules/binnacles/presentation/controllers/binnacles.controllers.ts`

```
@Controller('binnacles')
@Roles(Role.estandar)
export class BinnaclesController {
  @Get('me') findPaginatedMe(...) { ... }
  @Get(':id') findById(...) { ... }
  @Put(':id') update(...) { ... }
}
```

El decorador limita todo el controlador al rol estandar. Además, los Handlers comparan `binnacle.userId` con el sub del token; por eso conocer el ID de otra bitácora no concede acceso. Se combinan autorización por rol y autorización por propiedad.

4.5 Campos de base de datos

Archivo: `prx-backend/prisma/schema.prisma`, modelo `Binnacle`

```
model Binnacle {
  id      Int      @id @default(autoincrement())
  userId  Int
  name    String
  content String
  tasks   Json?
  links   Json?
  status  Int      @default(1)
  updatedBy Int?
}
```

Sí existen campos para guardar tareas y enlaces. Ambos se persisten como JSON y son convertidos por `BinnaclePrismaMapper`. `userId` determina la propiedad; `status` implementa borrado lógico; `createdBy` y `updatedBy` conservan auditoría.

4.6 Endpoints

Método	Ruta	Responsabilidad
POST	<code>/prx/binnacles</code>	Crear bitácora propia.
GET	<code>/prx/binnacles/me</code>	Listar bitácoras propias con paginación.
GET	<code>/prx/binnacles/:id</code>	Consultar una bitácora propia.
PUT	<code>/prx/binnacles/:id</code>	Actualizar contenido, tareas y enlaces.
DELETE	<code>/prx/binnacles/:id</code>	Realizar eliminación lógica.

5 Autenticación y autorización

El autenticador cubre el ciclo completo de identidad: solicitud y confirmación de registro, inicio de sesión, renovación de tokens, consulta del usuario actual, cierre de sesión y recuperación o cambio de contraseña.

5.1 Flujo de inicio de sesión

1. El usuario envía correo o nombre de usuario y contraseña.
2. `LoginHandler` busca la cuenta por el tipo de identificador.
3. `BcryptService` compara la contraseña con el hash almacenado.
4. Se crea una `Session` con `user agent` e IP cuando están disponibles.
5. `JwtTokenService` firma un `access token` y un `refresh token` con secretos y vencimientos distintos.
6. El `refresh token` se guarda asociado a la sesión.
7. El frontend guarda los tokens, actualiza `AuthStore` y adjunta `Bearer` a las solicitudes.

Archivo: prx-backend/src/modules/auth/application/commands/login/login.handler.ts

```
const validPassword = await this.bcryptService.compare(
  password,
  user.passwordHash,
);
const accessToken = await this.jwtTokenService.generateAccessToken(payload);
const refreshToken = await this.jwtTokenService.generateRefreshToken(payload);
```

La contraseña nunca se compara como texto plano contra la base de datos. El access token autoriza llamadas breves; el refresh token permite renovar la sesión sin pedir nuevamente la contraseña.

5.2 Protección de la API

Archivo: prx-backend/src/shared/presentation/guards/jwt-auth.guard.ts

```
if (!authHeader || !authHeader.startsWith('Bearer ')) {
  throw new UnauthorizedException(AUTH_MESSAGES.UNAUTHORIZED);
}
const payload = await this.jwtService.verifyAsync(token, { secret });
if (payload.tokenType !== TokenTypeEnum.ACCESS) {
  throw new UnauthorizedException(AUTH_MESSAGES.INVALID_TOKEN);
}
```

JwtAuthGuard rechaza solicitudes sin Bearer, tokens vencidos, firmas inválidas y refresh tokens utilizados incorrectamente como access tokens. CurrentUser extrae sub para que los casos de uso apliquen propiedad y auditoría.

5.3 Autorización por roles

Archivo: prx-backend/src/shared/presentation/guards/roles.guard.ts

```
const requiredRoles = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
  context.getHandler(),
  context.getClass(),
]);
if (!user?.role || !requiredRoles.includes(user.role)) {
  throw new ForbiddenException(AUTH_MESSAGES.FORBIDDEN);
}
```

RolesGuard lee los roles declarados por @Roles y devuelve 403 cuando el usuario está autenticado pero no autorizado. En Angular, standardUserGuard evita navegar a Bitácora si el usuario actual no tiene rol estandar. El backend sigue siendo la autoridad definitiva.

5.4 Renovación automática en el frontend

auth.interceptor agrega Authorization: Bearer <token>. refresh.interceptor detecta respuestas 401, solicita un nuevo par de tokens y reintentla la petición original. AuthRefreshService centraliza una sola renovación compartida para evitar múltiples refresh simultáneos. error.interceptor limpia la sesión cuando la renovación no es posible.

5.5 Endpoints de autenticación

Endpoint	Acceso	Función
POST /auth/register-request	Público	Iniciar registro y generar código.
POST /auth/confirm-register	Público	Confirmar código y crear cuenta.

Endpoint	Acceso	Función
POST /auth/login	Público limitado	Autenticar; máximo 3 intentos por minuto.
POST /auth/refresh	Público con refresh token	Rotar refresh token y emitir access token.
POST /auth/forgot-password	Público	Iniciar recuperación.
POST /auth/reset-password	Público con código	Restablecer contraseña.
PATCH /auth/change-password	JWT	Cambiar contraseña autenticada.
POST /auth/logout	JWT	Revocar token o sesión.
GET /auth/me	JWT	Recuperar identidad de la sesión.

5.6 Medidas de seguridad utilizadas

- Hash de contraseñas con bcrypt y salt rounds configurados.
- Secretos diferentes para access token y refresh token.
- Validación explícita del tipo de token.
- Sesiones y refresh tokens persistidos con revocación y expiración.
- Rotación del refresh token durante la renovación.
- Rate limiting en login.
- Guards en frontend para experiencia de navegación y guards en backend para seguridad real.
- DTOs y ValidationPipe para rechazar datos inválidos o campos inesperados.

6 Aplicación de los principios SOLID

SOLID no se demuestra por nombrar carpetas, sino mostrando cómo una modificación queda localizada. En PRX hay ejemplos claros y también aspectos que pueden seguir mejorándose.

Principio	Aplicación en PRX	Ejemplo defendible
S Responsabilidad única	Componentes muestran UI; Facades coordinan; Handlers aplican casos de uso; repositorios persisten.	BinnacleDetailPage no ejecuta Prisma y PrismaBinnacleRepository no decide navegación.
O Abierto cerrado	Los contratos permiten sustituir implementaciones sin cambiar al consumidor.	BinnacleRepository puede tener otra implementación además de Prisma.
L Sustitución de Liskov	Las implementaciones respetan el contrato abstracto esperado por los Handlers.	PrismaNoteRepository devuelve NoteEntity y conserva la semántica del repositorio.
I Segregación de interfaces	Los contratos están separados por módulo y propósito.	NoteApiContract no obliga a implementar operaciones de autenticación o Bitácora.
D Inversión de dependencias	Los casos de uso dependen de abstracciones inyectadas, no directamente de Prisma.	UpdateBinnacleHandler recibe BinnacleRepository mediante @Inject.

Ejemplo de inversión de dependencias

Archivo: update-binnacle.handler.ts

```
constructor(
  @Inject(BinnacleRepository)
  private readonly binnacleRepository: BinnacleRepository,
) {}
```

El Handler conoce la operación que necesita, no la tecnología de base de datos. El módulo enlaza el contrato con PrismaBinnacleRepository. Esto mejora pruebas, reemplazo de infraestructura y mantenimiento.

Evaluación honesta de SOLID

Fortaleza: la separación por capas, CQRS, facades, contratos y repositorios ofrece una aplicación clara de SRP y DIP.

Mejora: parseTasks y parseLinks están duplicados conceptualmente entre módulos; una política de validación compartida podría reducir repetición sin mezclar dominios.

Mejora: algunos componentes de página todavía coordinan muchas acciones de UI; conviene extraer componentes cuando aumente su complejidad, no antes.

7 Patrones de diseño utilizados

Patrón	Dónde aparece	Qué problema resuelve
Arquitectura por capas	Frontend y backend por presentación, aplicación, dominio e infraestructura.	Evita acoplar interfaz, reglas y persistencia.
CQRS	Commands, Queries, Handlers, CommandBus y QueryBus.	Separa lectura de escritura y enfoca cada caso de uso.
Repository	NoteRepository, BinnacleRepository y sus implementaciones Prisma.	Oculto detalles de base de datos al dominio.
Facade	NoteFacade, BinnacleFacade y AuthFacade.	Entrega una API simple a las páginas y coordina Store más servicio HTTP.
Mapper	NoteResponseMapper, Prisma mappers y AuthResponseMapper.	Convierte entre registros, entidades y DTOs sin filtrar formatos entre capas.
Guard	authGuard, JwtAuthGuard, RolesGuard y standardUserGuard.	Centraliza decisiones de acceso y evita repetir validaciones en rutas.
DTO	CreateNoteRequestDto, UpdateBinnacleDto y DTOs de Auth.	Define y valida el contrato de entrada.
Store reactivo	Stores Angular basados en signals.	Mantiene loading, datos y errores compartidos de forma predecible.
Soft delete	Campo status en Note y Binnacle.	Conserva auditoría sin mostrar registros eliminados.

Por qué CQRS es apropiado

Crear, actualizar y eliminar cambian el estado y se modelan como Commands. Listar o consultar por ID no cambia datos y se modela como Queries. Cada Handler tiene una entrada específica y una responsabilidad clara. Para este proyecto, CQRS también facilita ubicar reglas de permisos en el caso de uso correspondiente.

Por qué se usa Facade en Angular

La página no necesita conocer detalles del HttpClient ni cómo se administra loading. Por ejemplo, NoteFacade llama a NoteApi, normaliza respuestas paginadas y actualiza NoteStore. Esto reduce acoplamiento entre UI e infraestructura y permite que varias páginas reutilicen el mismo flujo.

8 Calidad, puntualidad y evidencias

8.1 Puntualidad

La fecha límite informada es el 23 de septiembre de 2026 a las 23:59 y este informe fue preparado el 22 de septiembre. La evidencia técnica puede presentarse antes del vencimiento. Sin embargo, la puntualidad de la entrega final depende de realizar el envío en la plataforma dentro del plazo; el código por sí solo no demuestra el momento del envío.

8.2 Calidad técnica comprobada

Comprobación	Resultado	Interpretación
Compilación Angular	Aprobada	ng build generó el bundle correctamente.
Servidor frontend	Activo	Disponible en http://127.0.0.1:4200/.
Servidor backend	Activo	Escucha en el puerto 3000.
Ruta protegida	401 sin token	GET /prx/binnacles/me respondió como recurso protegido.
Persistencia Bitácora	Verificada	GET y PUT fueron probados y se comprobó el cambio de tareas y enlaces.
Advertencia de build	No bloqueante	El SCSS de create-note supera el presupuesto por 818 bytes.

8.3 Criterios de calidad presentes

- Validación de formularios en frontend y DTOs en backend.
- Mensajes de error y estados loading para evitar dobles envíos.
- Paginación para no cargar todos los registros.
- Borrado lógico y campos de auditoría.
- URLs externas abiertas con rel noopener noreferrer.
- Comprobaciones de propietario o membresía antes de leer y modificar datos.
- Límites de tareas, enlaces, archivos y longitudes de texto.

8.4 Riesgos y mejoras antes de declarar terminado

Prioridad	Acción	Razón
Alta	Agregar búsqueda de notas.	Es parte explícita de HU 02.
Alta	Agregar tareas al formulario de creación de nota.	Cierra el alcance explícito de HU 03.

Prioridad	Acción	Razón
Alta	Construir el dashboard autenticado.	Cierra HU 01.
Media	Añadir pruebas unitarias y de integración.	Reduce regresiones en permisos y parseo JSON.
Baja	Reducir el tamaño del SCSS de creación de notas.	Elimina la advertencia del presupuesto Angular.

9 Guion para la presentación y defensa

Duración sugerida: 10 a 12 minutos. Cada afirmación debe acompañarse con una pantalla o archivo concreto.

Tiempo	Qué mostrar	Qué explicar
1 min	Objetivo y backlog	Tres historias, 18 puntos y alcance real alcanzado.
2 min	Arquitectura	Capas, flujo Angular a NestJS y separación CQRS.
3 min	Notas	Listado, creación, detalle, enlaces, archivos, tareas y guardado inmediato.
2 min	Autenticación	Login, JWT, refresh, sesión, guards y roles.
2 min	Bitácora	Ver separado de Editar, tareas marcables y propiedad por usuario.
1 min	SOLID y patrones	Ejemplos concretos de SRP, DIP, Repository, Facade y CQRS.
1 min	Calidad y cierre	Build, API protegida, brechas y próximos pasos.

Demostración recomendada

1. Iniciar sesión con un usuario estándar y mostrar que se recupera la sesión.
2. Abrir Notas y enseñar tarjetas, vista previa y paginación.
3. Crear una nota con contenido, enlace y archivo o imagen.
4. Abrir el detalle, marcar una tarea sin editar y comprobar que permanece tras recargar.
5. Entrar en Editar, cambiar contenido o enlaces y guardar.
6. Abrir Bitácora desde un usuario estándar, pulsar Ver y después Editar.
7. Marcar una tarea en la vista de Bitácora y recargar para evidenciar persistencia.
8. Cerrar con los archivos de Handler, Repository y Guard para explicar la arquitectura.

Mensaje de apertura sugerido

En este sprint construimos la base de gestión de notas y consolidamos la autenticación que protege los módulos. La solución no se limita a pantallas: conecta formularios Angular, casos de uso CQRS, control de permisos y persistencia Prisma. Además, adelantamos Bitácora con tareas y enlaces. Vamos a mostrar primero el flujo funcional y luego las decisiones de diseño que permiten mantenerlo.

Mensaje de cierre sugerido

El resultado demuestra separación de responsabilidades, seguridad por token, rol y propiedad, y persistencia completa de notas y bitácoras. El Sprint 1 está avanzado pero todavía requiere cerrar dashboard, búsqueda de notas y tareas durante la creación. Esas brechas están identificadas y tienen una ruta técnica concreta.

10 Preguntas probables y respuestas

¿Dónde se aplicó SOLID?

Principalmente en la separación entre páginas, Facades, Handlers y repositorios. UpdateBinnacleHandler depende del contrato BinnacleRepository, no de Prisma directamente, lo que evidencia inversión de dependencias.

¿Qué patrón es el más importante?

CQRS organiza cada lectura y escritura como un caso de uso independiente. Repository desacopla esos casos de uso de la base de datos y Facade simplifica el consumo desde Angular.

¿Por qué tareas y enlaces son JSON?

Son colecciones pequeñas, propias de una nota o bitácora, que normalmente se leen y escriben junto con su registro. JSON reduce tablas auxiliares. Si se necesitara consultar tareas globalmente, asignarlas a usuarios o auditarlas individualmente, convendría normalizarlas.

¿Cómo se evita que un usuario edite datos ajenos?

El JWT aporta el userId mediante sub. Los Handlers comparan ese valor con el propietario o verifican membresía. En Bitácora también se exige el rol estandar.

¿Por qué hay guards en frontend y backend?

El frontend mejora la experiencia y evita navegación inválida; el backend es la barrera de seguridad porque no puede confiar en el navegador.

¿Qué ocurre cuando vence el access token?

El interceptor detecta el 401, usa el refresh token, guarda el nuevo par y reintenta la solicitud. El backend rota el refresh token y comprueba que la sesión siga activa.

¿El Sprint 1 está terminado?

No completamente. Las funciones centrales existen, pero faltan dashboard autenticado, búsqueda de notas y tareas en la creación inicial. Se debe presentar como avance parcial con brechas identificadas.

¿Qué se probaría primero?

Permisos, parseo de tareas y enlaces, rotación de refresh token y persistencia al marcar tareas, porque son puntos con mayor impacto funcional y de seguridad.

11 Conclusiones y trabajo pendiente

PRX dispone de una base técnica coherente: Angular consume una API NestJS modular, los casos de uso están separados mediante CQRS, los repositorios encapsulan Prisma y los accesos se controlan con JWT, sesiones, roles y propiedad. Notas y Bitácora comparten una experiencia consistente para consultar, editar, manejar enlaces y completar tareas.

La defensa debe centrarse en evidencia ejecutable y en decisiones concretas de diseño. La mejor postura ante la evaluación es mostrar lo que funciona, explicar por qué está construido de esa manera y reconocer con precisión lo que falta.

Para cerrar Sprint 1: implementar dashboard autenticado, búsqueda de notas y tareas en el flujo de creación.

Para elevar calidad: incorporar pruebas automatizadas de permisos, autenticación, JSON y persistencia inmediata.

Avance adicional: Bitácora ya incorpora CRUD, enlaces, tareas, vista separada de edición y autorización por rol y propiedad.

Anexo Inventario de archivos principales

Notas frontend

- features/notes/domain/models/note.model.ts: Modelos de nota, tarea y enlace.
- features/notes/domain/requests/create-note.request.ts: Contrato de creación.
- features/notes/domain/requests/update-note.request.ts: Contrato de edición con tareas, enlaces y archivos.
- features/notes/application/facades/note.facade.ts: Coordinación de API y estado.
- features/notes/infrastructure/api/note.api.ts: Solicitudes HTTP y FormData.
- features/notes/infrastructure/store/note.store.ts: Estado reactivo del módulo.
- features/notes/presentation/pages/notes-page: Listado y tarjetas.
- features/notes/presentation/pages/create-note-page: Formulario de creación.
- features/notes/presentation/pages/note-detail-page: Consulta, tareas y edición.

Notas backend

- modules/notes/presentation/controllers/notes.controller.ts: Endpoints HTTP.
- modules/notes/application/commands: Casos de uso de escritura.
- modules/notes/application/queries: Casos de uso de lectura.
- modules/notes/domain/entities: Entidades del dominio.
- modules/notes/domain/repositories: Contratos de persistencia.
- modules/notes/infrastructure/persistence: Implementación Prisma.
- modules/notes/infrastructure/mappers: Conversión Prisma a dominio.

Bitácora

- prx-frontend/src/app/features/binnacles: Frontend completo del módulo.
- prx-backend/src/modules/binnacles: Backend completo del módulo.
- binnacle-detail-page.component.ts: Vista, edición y persistencia de tareas.
- binnacles.controllers.ts: CRUD protegido por rol estandar.
- prisma-binnacle.repository.ts: Persistencia y borrado lógico.

Autenticación

- prx-frontend/src/app/features/auth: Pantallas, Facade, API y Store de autenticación.
- prx-frontend/src/app/core/interceptors: Bearer, renovación y errores 401.
- prx-frontend/src/app/core/guards: Protección de rutas por sesión y rol.
- prx-backend/src/modules/auth: Registro, login, refresh, logout y contraseñas.
- jwt-auth.guard.ts: Verificación global del access token.
- roles.guard.ts: Autorización declarativa por rol.
- prisma/schema.prisma: Session, RefreshToken, VerificationCode y PasswordReset.